

Document Edition: 4.0 (Visual Master Release)

Frontend Architecture: Modular ES6 / Semantic HTML5 / Custom CSS3

Backend & Database: Google Cloud Firestore NoSQL & Firebase Auth

Security & Access: Server-Enforced RBAC & Declarative Security Rules

Hosting & Distribution: Multi-Region Global Edge CDN (Firebase / AWS)

NARI NIKETAN

COMPREHENSIVE SYSTEM ARCHITECTURE,
VISUAL FLOWCHARTS & IMPLEMENTATION
ANALYSIS

1. EXECUTIVE OVERVIEW & BUSINESS DOMAIN

Nari Niketan is an enterprise-grade, cloud-native E-Commerce platform built for modern Indian ethnic fashion (*Sarees, Suits, Lehengas, Kurtas, and Dupattas*). The platform bridges traditional artisanal handloom craftsmanship with modern web performance, empowering women artisans and providing direct-to-consumer access without intermediary commissions.

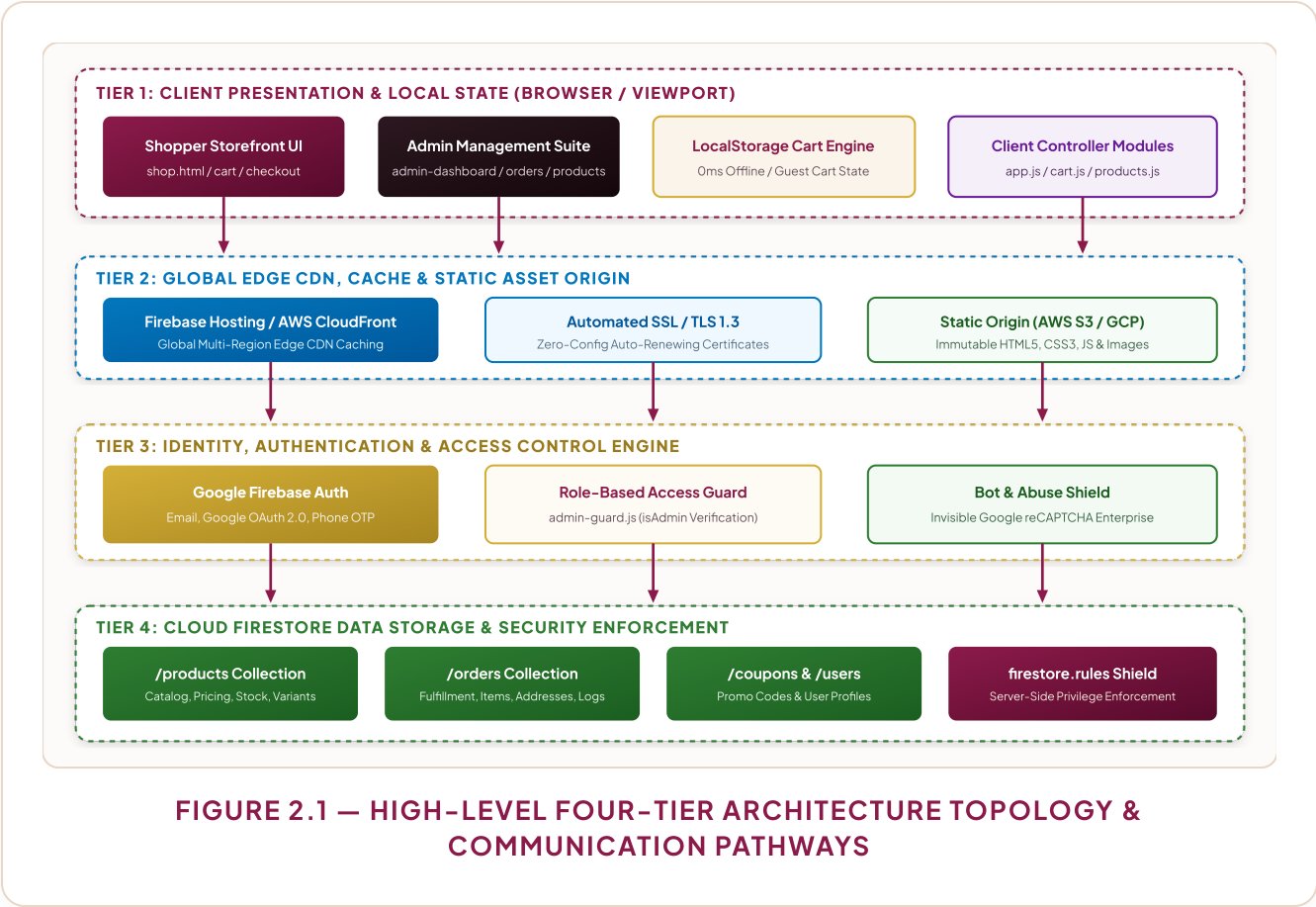
Core Value Proposition & System Capabilities

The platform provides shoppers with sub-second catalog discovery, real-time variant selections, offline-resilient cart storage, multi-channel checkout (COD, UPI QR, Cards), and **1-Click Self-Service Order Cancellation**. For store owners, it provides an autonomous administrative management suite with live revenue metrics, product catalog CRUD, order fulfillment trackers, and promotional coupon generators.

CAPABILITY PILLAR	CUSTOMER STOREFRONT EXPERIENCE	STORE ADMINISTRATOR OPERATIONS
Catalog & Inventory	Client-side multi-category filter, dynamic price slider, size/color variant chips, image galleries, live inventory availability indicators.	Modal-based Product CRUD (Add/Edit/Delete), high-resolution imagery URLs, pricing markdowns, instant stock unit counters.
Authentication & Identity	Unified multi-channel sign-in: Email/Password, 1-Click Google OAuth 2.0, SMS Phone OTP with anti-bot reCAPTCHA.	Dedicated Admin Portal Gate, Master Key activation (<code>NARINIKETAN2024</code>), server-enforced role verification.
Order Management & Fulfillment	Address validation, payment dispatcher (COD, UPI/QR, Cards), real-time order tracking, and 1-Click Order Cancellation .	Real-time sales revenue metrics, pending order alerts, fulfillment stage updater (Pending → Shipped → Delivered), cancellation logs.
Promotions & Discounting	Live promo code validator, active coupon carousel (<code>NARI20</code>), instant subtotal markdown, GST breakdown.	Coupon creator (Percentage / Flat discount), minimum order requirements, activation toggles, lifetime usage stats.

2. VISUAL SYSTEM ARCHITECTURE & MULTI-TIER DIAGRAMS

The Nari Niketan platform is built on a **decoupled, cloud-native multi-tier architecture** designed for high availability, zero server maintenance, and sub-100ms response times. Below is the detailed vector architecture diagram illustrating the interaction between the Client, Edge CDN, Authentication Gateway, and Cloud Firestore Data Tier:



3. TECHNOLOGY STACK & DETAILED RATIONALE ("WHY THIS TECH VS ALTERNATIVES")

Every technology in the Nari Niketan architectural stack was selected through rigorous benchmarking of performance, scalability, security, operational overhead, and long-term cost:

3.1 Frontend UI Layer: Semantic HTML5 + Vanilla CSS3 + Vanilla ES6 JavaScript

CHOSEN ARCHITECTURE

✓ Chosen: Semantic HTML5 + Vanilla ES6 JS

- **0ms Hydration Latency:** Immediate First Contentful Paint (FCP) and Time to Interactive (TTI) without JavaScript compilation delay.
- **Zero Dependencies:** No vulnerable npm packages, webpack bundler rot, or breaking framework lifecycle updates.
- **Extreme Network Efficiency:** Ultra-lightweight payloads ideal for mobile users across diverse 3G/4G/5G Indian mobile networks.
- **Long-term Maintainability:** Native web standards run reliably for decades without ecosystem depreciation.

✗ Alternative Considered: React / Next.js / Angular / Vue

- **Hydration Overhead:** 200KB–600KB+ framework runtime bundle that blocks main-thread CPU execution on mobile devices.
- **Complex Build Pipelines:** Requires Node.js build workers, Babel transpilation, and frequent package maintenance.
- **Server Costs:** Next.js SSR requires continuous Node server runtimes or costly Edge compute workers.

3.2 Database & Real-Time Data Tier: Google Cloud Firestore (NoSQL)

CHOSEN ARCHITECTURE

✓ Chosen: Cloud Firestore

- **Real-Time Sub-100ms Queries:** Document-based structure allows instant synchronization of product inventory and order states.
- **Serverless Auto-Scaling:** Automatically scales from zero to tens of thousands of concurrent shoppers with 0 server configuration.
- **Declarative Security Rules:** Database-level access control enforced via `firestore.rules` without requiring custom API boilerplate.
- **Multi-Region High Availability:** 99.999% SLA with automatic cross-zone data replication.

✗ Alternative Considered: Relational SQL (PostgreSQL / MySQL) / MongoDB

- **Infrastructure Overhead:** Requires dedicated database instances (AWS RDS / GCP Cloud SQL), VPC peering, connection poolers, and OS patching.
- **API Server Required:** Cannot connect directly from browser securely without writing an intermediary backend API service (Node/Express).
- **Fixed Monthly Cost:** Baseline database server costs even during low-traffic periods.

3.3 Identity & Authentication Engine: Google Firebase Authentication

CHOSEN ARCHITECTURE

✓ Chosen: Firebase Auth

- **Multi-Channel Identity:** Built-in support for Email/Password, 1-Click Google OAuth, and SMS Phone OTP verification.
- **Bot & Abuse Prevention:** Invisible Google reCAPTCHA integration stops SMS OTP fraud and credential stuffing.
- **Seamless Token Integration:** Issues cryptographically signed JWT tokens directly recognized by Firestore security rules.
- **Zero Security Liability:** Passwords and credentials never touch custom application servers; fully SOC 2 / GDPR compliant.

✗ Alternative Considered: Custom Node.js JWT Auth / Auth0

- **Custom JWT:** High security vulnerability risk (bcrypt salting, token refresh rotation, XSS/CSRF token storage, SMS gateway maintenance).
- **Auth0 / Okta:** Prohibitive enterprise pricing tiers and complex redirects that introduce friction to the checkout flow.

3.4 State Management & Cart Storage: Client LocalStorage Engine

CHOSEN ARCHITECTURE

✓ Chosen: Synchronous LocalStorage Cart Engine

- **Oms Instant Feedback:** Cart additions, quantity updates, and coupon recalculations reflect instantly on the UI.
- **Guest Shopping Support:** Shoppers can build their cart before creating an account or logging in.
- **Zero Server Load:** Eliminates database read/write load for temporary shopping carts until actual checkout placement.

✗ Alternative Considered: Server-Side Redis / Database Sessions

- **Network Latency:** Every cart click requires a round-trip network request, causing sluggish responsiveness.
- **Database Bloat:** Millions of abandoned cart session records requiring automated TTL cleanup workers.

4. END-TO-END USER JOURNEYS & SEQUENCE FLOWCHARTS

4.1 Customer Order Lifecycle & 1-Click Cancellation Sequence Flow

The vector sequence diagram below illustrates the exact step-by-step data exchange during item selection, local cart calculation, Firestore order creation, administrative fulfillment dispatch, and customer self-service cancellation:

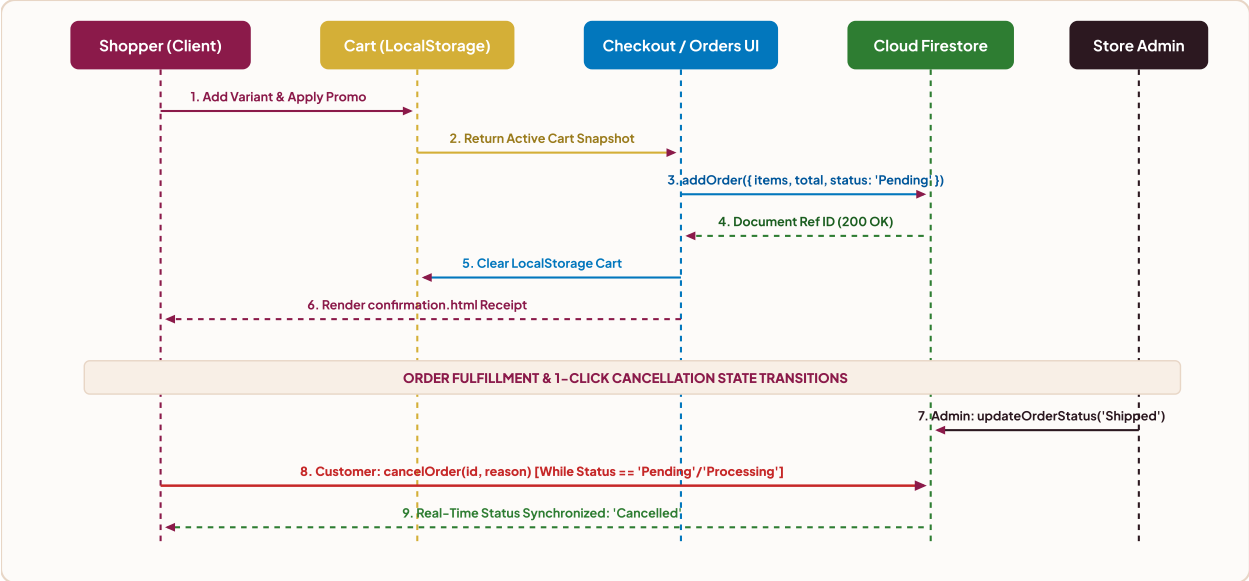


FIGURE 2.2 — SEQUENCE DIAGRAM: END-TO-END ORDER CREATION, FULFILLMENT & 1-CLICK CANCELLATION LIFECYCLE

5. DATABASE ENTITY-RELATIONSHIP (ER) SCHEMA & DATA MODEL

The Firestore NoSQL database utilizes a high-throughput, normalized document schema organized into four core collections:

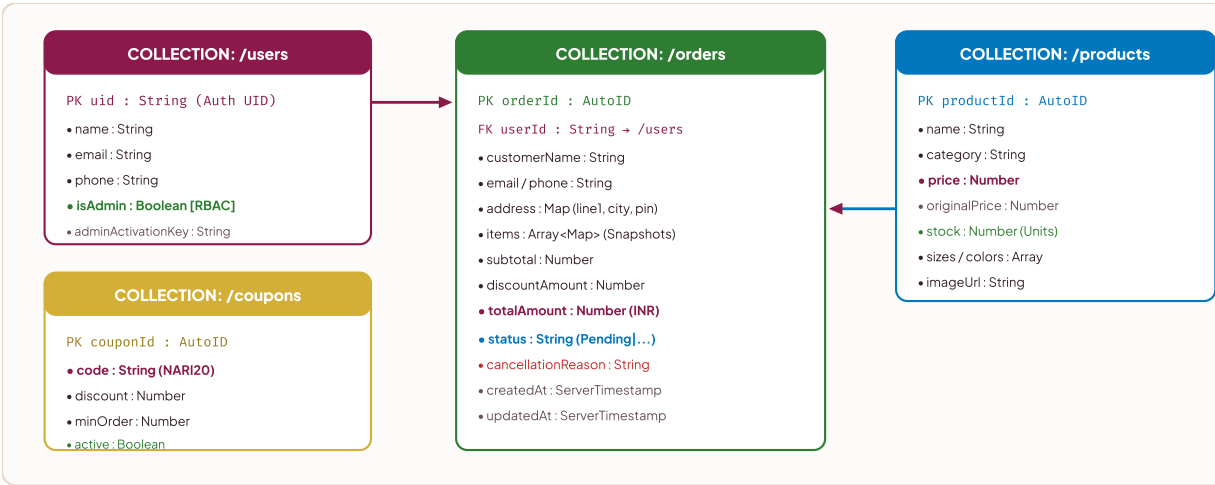


FIGURE 2.3 — CLOUD FIRESTORE DOCUMENT ENTITY-RELATIONSHIP (ER) SCHEMA MODEL

6. AUTHENTICATION, SECURITY & RBAC GUARD ENGINE

6.1 Server-Enforced Security Policies (`firestore.rules`)

```

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function isAuthenticated() {
      return request.auth != null;
    }

    function isAdmin() {
      return isAuthenticated() &&
        exists(/databases/${database}/documents/users/${request.auth.uid}) &&
        get(/databases/${database}/documents/users/${request.auth.uid}).data.isAdmin();
    }

    // 1. PRODUCTS: Public Read; Admin-only Write/Edit/Delete
    match /products/{productId} {
      allow read: if true;
      allow write, create, update, delete: if isAdmin();
    }

    // 2. ORDERS: Customer can create & read their own; Customer can only cancel; Admin can manage all
    match /orders/{orderId} {
      allow create: if isAuthenticated() && request.resource.data.userId == request.auth.uid;
      allow read: if isAuthenticated() && (resource.data.userId == request.auth.uid || isAdmin());
      allow update: if isAuthenticated() && (
        isAdmin() ||
        (resource.data.userId == request.auth.uid && request.resource.data.status == 'CANCELLED')
      );
      allow delete: if isAdmin();
    }

    // 3. COUPONS: Public Read; Admin-only Write
    match /coupons/{couponId} {
      allow read: if true;
      allow write, create, update, delete: if isAdmin();
    }

    // 4. USERS: Read/write own profile; Prevent self-elevation of isAdmin unless Matched by Admin
    match /users/{userId} {
      allow read: if isAuthenticated() && (request.auth.uid == userId || isAdmin());
    }
  }
}

```

```
allow create: if isAuthenticated() && request.auth.uid == userId;
allow update: if isAuthenticated() && (
  isAdmin() ||
  (request.auth.uid == userId && (!request.resource.data.diff(resource.data).dataChanges().has('delete')))
  (request.auth.uid == userId && request.resource.data.adminActivationKey == resource.data.adminActivationKey)
);
allow delete: if isAdmin();
}
}
}
```

7. COMPLETE FILE HIERARCHY & COMPONENT BREAKDOWN

FILE PATH	CATEGORY	ARCHITECTURAL RESPONSIBILITY
index.html	Storefront	Homepage, Hero banner carousel, Category highlights, Featured items grid.
shop.html	Storefront	Full product catalog with sidebar filters (Categories, Price range, Sizes, Sorting).
product.html	Storefront	Product detail view with image gallery, variant selectors, stock badge, and related items.
cart.html	Storefront	Cart management, quantity adjustments, coupon validation, price summary breakdown.
checkout.html	Storefront	Delivery address entry, payment option selection, order submission to Firestore.
confirmation.html	Storefront	Order success receipt with step-by-step progress indicator and tracking link.
my-orders.html	Storefront	Customer dashboard displaying order history and 1-Click Order Cancellation modal .
login.html / register.html	Auth	Unified login & registration with Email, Google OAuth, and Phone OTP modes.
admin-login.html	Admin	Admin gate with Master Key activation system.
admin-dashboard.html	Admin	Executive revenue metrics, total orders counter, pending order alerts, recent orders table.
admin-products.html	Admin	Product catalog CRUD with modal forms for adding/editing items and managing stock.
admin-orders.html	Admin	Order fulfillment pipeline, customer shipping details inspection, status updater.
admin-coupons.html	Admin	Promo code creation, discount configuration, minimum order limits, activation toggles.
js/firebase-config.js	Core JS	Firebase SDK initialization, project configuration, and auth persistence.

FILE PATH	CATEGORY	ARCHITECTURAL RESPONSIBILITY
js/store.js	Core JS	Centralized Cloud Firestore Data Access Object (DAO) with error handling and query logic.
js/auth.js	Core JS	Authentication controller for Email, Google OAuth, and Phone OTP with reCAPTCHA.
js/cart.js	Core JS	Client-side localStorage shopping cart engine and discount calculations.
js/products.js	Core JS	Catalog rendering and dynamic multi-criteria filter controller.
js/admin.js	Core JS	Admin controllers for product CRUD, order transitions, and coupon actions.
js/admin-guard.js	Core JS	Client-side route protection verifying admin status before rendering admin views.
js/app.js	Core JS	Global app lifecycle, navbar counter badge, toast notification engine.
css/style.css	Styles	Global Burgundy & Gold design system, typography, animations, responsive grids.
css/shop.css	Styles	Product cards, galleries, filters, cart layout, order cards.
css/auth.css	Styles	Auth cards, OTP input grids, mode switchers, reCAPTCHA containers.
css/admin.css	Styles	Admin layout, sidebar navigation, metric cards, data tables, modals.

8.1 Deploying to Google Firebase Hosting

8.2 Deploying to Amazon Web Services (AWS S3 + CloudFront)

◀ ▶